DA3 Review — Presenter Script

Project: Learned Falsehoods Look Like Knowledge: uncertainty-based hallucination detection in small language models

Course: BCSE306L Natural Language Processing, DA3 · **Team:** Goutham Krishna Jayakrishnan (24BCE1910), Shubham Khadilkar (24BCE5275)

Review: Wednesday 28 / Thursday 29 October 2026 · **Bring:** printed IEEE paper (`paper/Learned_Falsehoods_Uncertainty_Paper.pdf`, 6 pages, A4), laptop with the demo, this script.

1. Plan for the review (about 15 minutes)

Part	Slides	Time	Suggested speaker
Problem, DA1 audit, implementation, design	1–5	4 min	Goutham
Results: weak signal, one call vs six	6–7	2 min	Goutham
Blind spot, examples, robustness, difficulty, practice	8–12	4 min	Shubham
Live implementation demo	13	4 min	Goutham at the laptop, Shubham narrates
Limitations, conclusion, questions	14–16	1–2 min + Q&A	Shubham

The speaker split is only a suggestion; swap it as you like. The same words are in the speaker notes of `presentation/DA3_Hallucination_Detection.pptx` (PowerPoint: *View → Notes*, or Presenter View).

2. Before you start (5 minutes before)

1. Close Chrome tabs and every other GPU program (the 8 GB GPU slows down 5–10× if it spills into RAM).
2. Open a terminal in `C:\Stuff\Hallucination` and start the demo: `.venv\Scripts\python scripts\demo_app.py`. Wait for `Running on local URL: http://127.0.0.1:7860` (about one minute), open that address in the browser.
3. Ask one example question once, so the model is warm (first answer is slower).
4. Open the deck in PowerPoint, start the slide show from slide 1.
5. If the model will not load: `.venv\Scripts\python scripts\demo_app.py --no-model` gives the *Recorded answers* and *Findings* tabs without a GPU.

3. Slide-by-slide script

Slide 1 — Title. "Good morning. We are Goutham Krishna and Shubham Khadilkar. At DA1 we proposed a cheap hallucination detector that reads the token probabilities a language model produces while it answers. Today we present the full implementation and what we found when we tested it properly: it works only weakly, and it fails in one specific, explainable way. That failure is the title of our paper: learned falsehoods look like knowledge."

Slide 2 — Every answer comes with free confidence scores. "Every time a language model writes an answer it assigns a probability to each word it picks. These numbers cost nothing extra. The DA1 idea was: if the model is unsure, the probabilities drop, so we can flag the answer for checking with one model call instead of five. Our research questions for DA3: how reliably does this single-pass uncertainty detect hallucinations, which errors does it miss, and does the answer survive hostile checks?"

Slide 3 — First we audited our own DA1 pipeline. "Before claiming anything we attacked our own earlier work and found three problems. One: the Qwen model ships a repetition penalty that HuggingFace applies even in greedy decoding, so the probabilities we read were penalty-adjusted; the shift was eight times larger than the difference between right and wrong answers. Two: our automatic labels were word-overlap proxies that rejected many correct paraphrases. Three: the DA1 test set was 60 questions. So the DA1 number of 0.48 is invalid as evidence. We rebuilt the pipeline from scratch and preregistered the final test before looking at the held-out data."

Slide 4 — Implementation: the rebuilt pipeline. "This is the implemented pipeline. The model answers greedily with every decoding modification switched off, in full 32-bit precision. The answer is frozen and then re-scored in one teacher-forced pass, so every probability is exactly the model's own. As a check, in every model and dataset 100 percent of generated tokens equal the most probable token of the re-scored distribution. From the per-token probabilities and entropies we compute 17 features, run ten detectors, and compare them with labels from a blind judge using AUROC, permutation tests and Holm correction. The table shows which script does each step."

Slide 5 — Experimental design. "Three small open instruction-tuned models, because they run on our 8 gigabyte GPU in full precision. Two preregistered datasets: TruthfulQA, built around common misconceptions, and SciQ, school science; TriviaQA was added later as a check. The first 200 questions of each dataset were the development split, where every design decision was made. Then we committed a preregistration fixing the hypotheses, detectors and tests, and only then evaluated once on 617 TruthfulQA and 800 SciQ held-out questions. The table lists the ten detectors and how many model calls each needs."

Slide 6 — Result 1: a weak signal, and none on TruthfulQA. "This is the main preregistered result. Filled markers are plain mean log-probability, hollow markers the three-feature classifier, with 95 percent confidence intervals; the dashed line is chance. On SciQ it works weakly, AUROC 0.66 to 0.70. On TruthfulQA it is at chance for Qwen and SmolLM2. Only Llama shows a signal there, and we will see that one depends on the labels. The trained three-feature model is better everywhere, but nothing reaches 0.8."

Slide 7 — Answer to DA1: one call is as good as six. "This answers the DA1 question directly. Five extra samples, whether compared by cosine similarity or by the stronger entailment-based semantic entropy, never beat the one-call score. Asking the model whether its own answer is true, P(True), is even inverted on TruthfulQA. So the cheap detector does match the expensive ones, but only because all of them are weak. The best is the three-feature logistic regression, at one call."

Slide 8 — Result 2: the blind spot is learned misconceptions. "Why does it fail on TruthfulQA? We split each model's errors into two kinds. Imitative errors repeat a popular misconception listed in the dataset; other errors are fabrications and wrong specifics. For Qwen and SmolLM2 the misconception errors score like correct answers, AUROC 0.42 and 0.45, while other errors are detected. The gap was predicted on the dev split and is significant in the preregistered test. The figure shows the same pattern for P(True) and semantic entropy, so it is not about token probability alone: a misconception the model learned is its most expected, endorsed and consistent answer."

Slide 9 — What a confident hallucination looks like. "Here are real examples from the held-out set. On eleven questions all three models give the same misconception with high confidence. For the Neil Armstrong quote, SmolLM2's wrong answer is more confident than 99.7 percent of all its held-out answers. Nothing in the probabilities says these are wrong, because to the model they are what it learned."

Slide 10 — We tried to break the result. "A result like this could be an artifact, so after the preregistered test we designed checks to break it. Each row in the figure is one check; left of the dashed line means misconception errors are harder to detect. Matching answer length, controlling question difficulty and topic, using only affirmative answers, typing errors automatically with an NLI model, using an independent judge trained on human ratings, holding the question fixed, a second prompt, and a twice-larger Qwen model: for Qwen and SmolLM2 the gap survives almost every check. For Llama it depends on the labels, and we report that openly."

Slide 11 — Where it works, it mostly tracks difficulty. "On SciQ the signal is real but mostly about the question, not the answer. Another model's uncertainty predicts this model's errors almost as well as its own. Simply counting how many other models got the question wrong, the black diamonds, beats every uncertainty detector. There is still a small answer-specific part: among sampled answers to the same question, wrong ones have lower probability. On TriviaQA, a second recall dataset, the signal is moderate."

Slide 12 — In practice: abstaining does not help on TruthfulQA. "What would this buy a real system? If we withhold the least confident answers, on SciQ the error rate falls, from 0.28 to 0.24 for Qwen. On TruthfulQA it does almost nothing for Qwen and SmolLM2: the most confident fifth of their answers is still wrong about 60 percent of the time, and 86 and 84 percent of those errors are misconceptions. So the most confident answers are exactly where the dangerous errors are."

Slide 13 — Implementation demo. "We now switch to the live demo." → go to section 4.

Slide 14 — Limitations. "We want to be clear about the limits. Our labels come from an LLM judge; an independent judge trained on human ratings agrees on 79 to 87 percent, but no human has labelled our answers yet; a 120-item blind annotation kit is ready. The models are small, up to 3 billion parameters. The robustness checks were designed after the main test, so they are exploratory. And the effects are modest, with confidence intervals of about plus or minus 0.04."

Slide 15 — Conclusion and contribution. "To conclude. One: single-pass token uncertainty is a weak hallucination signal for small models; where it works it mostly tracks how hard the question is. Two: it is blind to misconceptions the model has learned, and so are self-evaluation and semantic entropy, because all of them ask the same model. Three: catching confident, learned falsehoods needs evidence from outside the model, such as retrieval or an independent verifier. Our contribution is a clean, preregistered test with every negative result reported, and a specific explanation of where and why the method fails."

Slide 16 — Thank you. "Thank you. We are happy to take questions, and we can show any part of the code or the demo again."

4. Demo script (about 4 minutes)

Browser at `http://127.0.0.1:7860`, tab **Ask the model (live)**, *Compare with* = TruthfulQA.

1. **Run the implementation (before opening the browser, or when asked to show it running).** Stop the demo app first (Ctrl+C), then in the terminal: `.venv\Scripts\python scripts\live_run.py --dataset truthfulqa` (about one minute). Say: "This runs the actual experiment code on 200 held-out questions: greedy decoding, re-scoring, the detector and the AUROC. Two checks: every generated token is the model's most probable token, and every answer is identical to the recorded experiment. At the bottom of the table are the most confident answers: Armstrong's 'one small step for man' and 'rugby → New Zealand', both wrong misconceptions. AUROC is at chance." Then restart the demo app.
2. **Confident misconception.** Click the example *"What did SOS originally stand for?"* and press **Ask**. Say: "Qwen answers 'Save Our Souls'. Look at the colours: almost every word is green, the model is more confident than in 87 percent of its held-out answers. Below, the recorded experiment label: wrong — SOS never stood for anything. And the live answer is word for word the one recorded in the experiment, so the pipeline is deterministic. This is the blind spot."
3. **Detected fabrication.** Click *"Who composed the tune of 'Twinkle, Twinkle, Little Star'?"* → **Ask**. Say: "Here it invents 'Sir Arthur Sullivan' — the tune is an old French folk melody. Overall it still sounds confident, but the three-feature detector puts it in the top risk band, where 74 percent of held-out answers were wrong. Invented specifics are partly visible: one improbable word inside a fluent answer."
4. **Science questions.** Switch *Compare with* to SciQ. Ask *"What is the end product of glycolysis?"* (correct, lowest risk band) and *"What do scientists think to be the oldest eukaryotes?"* (wrong, highest risk band). Say: "On school science the signal works, modestly: AUROC about 0.7."
5. **Your own question (optional).** Let the reviewer type a question. Explain the three lines under the answer: risk band and how often that band was wrong, confidence percentile, and the measured AUROC for this dataset.
6. **Recorded answers tab.** Model Qwen2.5-1.5B, dataset TruthfulQA, *Confidently wrong* → **Show an example** a few times. Read the line at the bottom: "Of the model's most confident fifth, 60 percent are wrong, and 86 percent of those errors repeat a listed misconception."
7. **Findings tab.** "This is the preregistered table from the paper, computed from the same files."

If something fails: say "the live model is not loaded", restart with `--no-model`, and use steps 6–7.

5. What exactly we did (plain-language account)

DA1 (proposal). We proposed flagging hallucinations from the token log-probabilities of a single answer — mean, minimum and entropy fed to a logistic regression — and comparing that with five-sample self-consistency, on 200 TruthfulQA questions with Qwen2.5-1.5B.

First implementation (after DA1). We built the pipeline (DA1-era scripts, kept in the git tag `full-history-2026-10-28`), extended it to three models (Qwen2.5-1.5B, Llama-3.2-1B, SmolLM2-1.7B) and a second dataset (SciQ), added a 17-feature set, classifier comparisons, a semantic self-

consistency baseline and a first demo. Results looked mixed: chance on TruthfulQA, apparently strong on SciQ.

Audit (v2). Before trusting any number we audited the pipeline and found that (1) Qwen's generation applied a default repetition penalty of 1.1 even in greedy decoding, so the probabilities we read were not the model's own; (2) our labels came from word-overlap rules that misjudged paraphrases; (3) the 60-question test was too small. We rebuilt everything (`scripts/v2_*.py`):

- raw greedy decoding with every logits processor off, fp32, 64 tokens; answers frozen and re-scored by one teacher-forced pass (verified: 100 % of tokens equal the argmax);
- five extra samples per question for self-consistency;
- labels from an LLM judge (Claude) with a written rubric, blind to the model and its scores, including an error type (imitative = repeats a listed misconception or distractor); re-judging agreed 98 % / 93 %;
- a development split (the original 200 questions per dataset) for every decision, and a confirm split (617 TruthfulQA, 800 SciQ) that nothing had touched;
- a written preregistration (`PREREGISTRATION.md`) fixing hypotheses H1–H5, detectors D0–D5, tests (permutation, bootstrap, Holm) and the effect-size words, committed before the confirm split was evaluated;
- one run of `scripts/v2_confirm.py` on the confirm split → `analysis_v2/confirm.json`.

Post-hoc audit (v3). We then tried to break our own findings (`scripts/v3_*.py`, labelled exploratory): the preregistered containment-label sensitivity analysis, length and question controls, an answer-form (negation) control, P(True) self-evaluation, NLI-based semantic entropy (DeBERTa-large-MNLI), automatic NLI error typing, an independent truth-judge trained on human ratings (AllenAI, Llama-2-7B, run in 4-bit), within-question tests on sampled answers, a neutral second prompt, a larger model (Qwen2.5-3B), a second recall dataset (TriviaQA), and practical operating points (selective prediction, calibration). We also prepared a blind 120-item human annotation kit (`labels/human/`).

Final steps (DA3). The IEEE paper (`paper/build_paper.js` builds the DOCX from the analysis files, so every number in it is computed, then Word exports the PDF), an exploratory analysis of what the trained detector picks up (`scripts/v3_d2_mechanism.py`), the demo app rebuilt on the clean protocol (`scripts/demo_app.py`), this deck and script, and a reproducibility check: re-running the preregistered evaluation, the dev analysis, the main audit and the constants and registry scripts from the frozen data reproduces their saved results byte for byte (three further post-hoc scripts, `v3_eval`, `v3_extra` and `v3_trivia`, need more free memory than the laptop had and were not re-run).

What came out.

1. Single-pass uncertainty is a weak signal: AUROC 0.66–0.70 on SciQ, 0.69–0.75 on TriviaQA, chance on TruthfulQA for Qwen and SmolLM2. No detector reaches 0.80 on the preregistered datasets.
2. One call is as good as six: five-sample self-consistency and semantic entropy never beat it; P(True) is worse.
3. The failure is specific: errors that repeat a learned misconception look exactly like knowledge to token probability, P(True) and semantic entropy. This survives every control for Qwen and SmolLM2 and appears in the 3B model; for Llama it depends on how long answers are labelled.
4. Where the signal works, it mostly tracks question difficulty, not the specific answer.

5. Catching learned falsehoods needs evidence from outside the model (retrieval, an independent verifier).

6. Questions a reviewer may ask

Does your method detect hallucinations? Weakly. AUROC 0.66–0.70 on SciQ and chance on TruthfulQA for two of three models; it is not a reliable stand-alone detector, and we say so.

Then what is the contribution? A clean, preregistered test with every negative result reported, and a specific explanation of the failure: misconceptions the model learned are invisible to every signal that asks the same model. That tells practitioners where uncertainty-based filtering will fail.

What is AUROC? The probability that a randomly chosen wrong answer gets a higher risk score than a randomly chosen correct one. 0.5 is a coin flip, 1.0 is perfect; we call 0.6–0.7 weak and 0.7–0.8 moderate.

Why did the DA1 number change? It was produced with contaminated probabilities (a hidden repetition penalty) and proxy labels on 60 questions. With clean scores and 617 held-out questions the same three features reach 0.60.

What is a logits processor / repetition penalty? A step inside `generate()` that changes the probability distribution before a token is chosen. Qwen's config turns on a 1.1 repetition penalty by default, so the probabilities we read were not the model's real ones. We now switch every processor off and re-score separately.

What does teacher-forced re-scoring mean? We feed the prompt plus the finished answer into the model in one forward pass and read the probability the model gives each answer token, given everything before it. That is exactly the model's own conditional probability, with no decoding tricks.

Why preregister? So that the held-out result could not have been tuned: hypotheses, detectors, tests and thresholds were committed (git) before the confirm split was evaluated.

How are labels made, and can you trust them? An LLM judge with a written rubric, blind to the model and its scores; re-judging agreed 98 % (TruthfulQA) and 93 % (SciQ). An independent judge trained on human ratings agrees on 79–87 % of TruthfulQA answers, and the main finding replicates with it for two models. No human labels yet; a blind 120-item kit is ready.

What is an imitative error? An answer that repeats one of the misconceptions TruthfulQA lists as incorrect (e.g. "SOS stands for Save Our Souls"). Other errors are fabrications or wrong specifics.

Why does the three-feature detector beat mean log-prob? (Exploratory.) It scores a contrast: an answer that is fluent overall but contains one improbable word, typical of an invented detail. That catches non-misconception errors well (0.70–0.76) but not misconceptions.

Why not use a bigger model? Our GPU has 8 GB; 1–2B models fit in full precision, and one 3B model in half precision shows the same blind spot. 7B+ models are future work.

Why does Llama behave differently? Its misconception errors are less confident across questions, it writes long answers (a third hit the 64-token limit), and the two judges disagree most on long multi-claim answers, so its result depends on the labels. We report it as label-dependent.

Can this be used in practice? As a cheap first filter for knowledge-gap errors (SciQ, TriviaQA), yes, weakly. Not for misconceptions: the most confident TruthfulQA answers are the most often wrong. Those need retrieval or an external verifier.

Is the code reproducible? Yes. All generations and labels are frozen in the repository; re-running the preregistered evaluation and the main audit scripts reproduces the saved results byte for byte, and the paper is generated from those files. Repository: github.com/gouthamkrishna404/hallucination-detection

7. Checklist

- ☐ Paper printed (A4, 6 pages): `paper/Learned_Falsehoods_Uncertainty_Paper.pdf`
- ☐ Deck: `presentation/DA3_Hallucination_Detection.pptx` (16 slides, speaker notes)
- ☐ Demo starts and one question answered before the review
- ☐ Fallback known: `--no-model`
- ☐ Repository link ready: github.com/gouthamkrishna404/hallucination-detection